

LLMs Embarqués sur Smartphone

Chronologie de l'État de l'Art · 2015–2025

Solutions Google (Gemini Nano, Flash, Lite) et alternatives open source (llama.cpp, MLC-LLM, Gemma)

Architecture

Avantage

× Limite

2015

2020

[12]

Optimisation

Distillation de connaissance

Hinton, Vinyals & Dean — NIPS Deep Learning Workshop

Un modèle étudiant compact apprend à imiter les distributions de probabilité (*soft targets*) d'un modèle enseignant plus grand, transmettant une connaissance plus riche que les simples étiquettes.

Avantages	Limites
• Transmission de connaissance riche sans ré-entraînement complet • Réduit le coût d'entraînement de 90 % (Apple Intelligence) • Compatible avec tous les modèles existants	• Nécessite un modèle enseignant de qualité disponible • Perte de capacités émergentes si compression trop agressive • Difficile à contrôler précisément les capacités transmises

2022

[1]

Modèle

GPT-3 — Few-Shot Learners

Brown et al. — NeurIPS 2020 · arXiv:2005.14165

175 milliards de paramètres. Démontre que les capacités émergentes (few-shot, zero-shot) apparaissent brusquement à partir d'un seuil de paramètres. Pose la question fondatrice : comment rendre ces capacités accessibles sur 6 Go de RAM ?

Avantages	Limites
• Définit le paradigme du few-shot learning sans fine-tuning • Capacités généralistes remarquables • Fondation théorique pour toute la compression LLM	• 175B params, plusieurs centaines de Go GPU • Totalement inutilisable sur mobile en l'état • Déclenche une course aux paramètres avant la compression

[7]

Optimisation

LLM.int8() — Quantification 8-bit

Dettmers, Lewis, Belkada & Zettlemoyer — NeurIPS 2022

Décomposition mixte : 0,1 % des valeurs extrêmes (outliers) maintenues en FP16, reste en INT8. Divise la mémoire par deux avec moins de 1 % de dégradation de qualité au-delà de 6,7B paramètres.

Avantages

Limites

- ÷2 mémoire avec <1 % de dégradation
- Pas de ré-entraînement nécessaire (post-training)
- Compatible avec tous les modèles Transformer

- INT8 insuffisant pour les appareils <8 Go RAM
- Overhead de 20 % en latence sur certains SoCs sans support matériel
- Peu adapté aux GPU mobiles Mali (pas de support natif)

[8]

Optimisation

GPTQ — Quantification INT4

Frantar, Ashkboos, Hoefler & Alistarh — arXiv:2210.17323

Minimisation d'erreur couche par couche. LLaMA 7B passe de 14 Go (FP16) à ~4 Go (INT4). Rend possible l'exécution sur flagship Android. Base du format GGUF.

Avantages

Limites

- -68,66 % de taille (LLaMA 3.2 3B validé Android)
- INT4 = sweet spot qualité/performance pour 3-7B
- Standard adopté par llama.cpp et HuggingFace

- Perte de 2-5 % de qualité vs FP16 sur raisonnement
- Compilation longue (plusieurs heures pour 7B)
- Instable pour les modèles <1B (INT2 dégrade fortement)

[11]

Optimisation

QLoRA — Fine-tuning quantifié

Dettmers, Pagnoni, Holtzman & Zettlemoyer — NeurIPS 2023

Combine quantification NF4 + adaptateurs LoRA bas-rang. Fine-tuning d'un modèle 65B sur un seul GPU A100 48 Go avec performances proches du full-precision. Adaptateurs LoRA chargeables dynamiquement.

Avantages

Limites

- Personnalisation sans ré-entraînement complet du modèle
- Adaptateurs LoRA légers (<100 Mo) par tâche
- Exploité par Google ML Kit GenAI pour Gemini Nano

- Performances légèrement inférieures au fine-tuning full-precision
- Nécessite des données de qualité pour chaque tâche
- Latence ajoutée lors du chargement d'un nouvel adaptateur

[4][9]

Framework

llama.cpp + Format GGUF

Georgi Gerganov — github.com/ggml-org/llama.cpp

Implémentation C/C++ zéro-dépendance. ARM NEON + AVX2/AVX512. Format GGUF (poids + tokenizer + métadonnées en un fichier). S'installe dans Termux sans root. Standard de facto avec >100 000 fichiers sur HuggingFace.

Avantages

Limites

- Compatible Android 7+ sur tout appareil ARM64
- Aucune restriction matérielle (pas d'appareil certifié requis)
- Écosystème massif : >100 000 modèles GGUF disponibles
- Actif, mis à jour très fréquemment

- GPU Mali (Samsung/MediaTek) : accélération quasi inutilisable
- Inférence CPU uniquement sur appareils Exynos/Dimensity (faible perf.)
- Pas d'interface graphique Android native intégrée
- Consommation mémoire moins optimisée que MLC-LLM sur GPU

[5]

Framework

MLC-LLM — Compilateur TVM

MLC AI Contributors — github.com/mlc-ai/mlc-llm

Compilateur TVM génère du code optimisé pour chaque cible matérielle. Supporte les GPU Mali via Vulkan (inaccessibles à llama.cpp). APK Android directement installable. 20-25 % plus rapide que llama.cpp sur Snapdragon 8 Gen 3.

Avantages

Limites

- Seul framework open source avec support GPU Mali (Vulkan)
- 20-25 % plus rapide que llama.cpp sur flagship
- APK Android directement installable, interface utilisateur intégrée
- Support multiplateforme (Android, iOS, Web, PC)

- Format de modèles compilés propriétaire (incompatible GGUF)
- Compilation du modèle nécessaire par cible (longue et technique)
- Moins de modèles disponibles que l'écosystème GGUF
- Interface CLI moins mature que llama.cpp

[15]Modèle

Gemini — Famille multimodale

Google DeepMind — arXiv:2312.11805

Première famille multimodale native (texte, image, audio, vidéo, code). Ultra : 90,0 % MMLU (niveau expert humain). **Nano-1** (1,8B) et **Nano-2** (~3,25B) : 79,6 % MMLU, déployés via AICore sur Pixel 8/9 et Galaxy S25.

Avantages	Limites
- Nano-2 : 79,6 % MMLU, surpasse des modèles 2× plus grands - Support multilingue excellent (hérité de Gemini) - Partagé entre toutes les apps via AICore (−72 % RAM système) - Multimodal natif (texte + image sur mobile)	- Uniquement sur appareils certifiés AICore (flagships récents) - Modèle non accessible directement (boîte noire propriétaire) - Quota d'inférence par application - Dépendance totale à Google pour les mises à jour

2024

[6]Modèle

Gemma — Open source Google

Gemma Team, Google DeepMind — arXiv:2403.08295

Version open source de Gemini sous licence commerciale permissive. Gemma 2 2B (distillé depuis 27B) rivalise avec GPT-3.5. Gemma 3 (2025) : variantes 1B, 4B, 12B avec support multimodal natif (vision).

Avantages	Limites
- Testable sur tout Android ARM64 (llama.cpp ou MLC-LLM) - Licence commerciale permissive (usage prod. autorisé) - Gemma 2 2B : excellent pour classification et NER - Gemma 3 : support vision natif embarqué (4B, 12B)	- MMLU de Gemma 2 2B (~52 %) inférieur à Phi-3 Mini - Raisonnement multi-étapes limité à cette taille - Performances moindres en français vs anglais (~10–15 %) - Gemma 12B trop lourd pour la plupart des smartphones

[14]Modèle

Phi-3 Mini — Qualité des données

Abdın et al. — Microsoft Research — arXiv:2404.14219

Rupture conceptuelle : la qualité des données compense la taille. Phi-3 Mini (3,8B) dépasse Mistral 7B sur MMLU (68,8 % vs 61,7 %) et atteint 82,5 % sur GSM8K. Données synthétiques distillées depuis GPT-4.

Avantages	Limites
- Meilleur raisonnement arithmétique parmi les modèles <4B (82,5 % GSM8K) - Licence MIT (réutilisation commerciale libre) ~3 Go RAM en Q4_K_M, viable sur tous les flagships - Excellent instruction-following général	- Pré-entraînement orienté anglais : ~15–20 % de baisse en français - Données synthétiques : biais GPT-4 potentiel - Moins performant sur les tâches de résumé multilingue - Surpassé par Apple Intelligence 3B (propriétaire) sur MT-Bench

[13]Modèle

Apple Intelligence — QAT 2-bit

Apple Machine Learning Research — arXiv:2507.13575

Modèle on-device ~3B compressé à 2 bits/poids (QAT). KV Cache Sharing : −37,5 % mémoire et TTFT. Architecture hybride avec Private Cloud Compute (garanties cryptographiques vérifiables).

Avantages	Limites
- Surpasse Phi-3 Mini, Mistral 7B, Gemma 7B et LLaMA 3 8B (MT-Bench) ~1,4 Go RAM grâce à la quantification QAT 2-bit - Confidentialité maximale : données jamais quittent l'appareil (PCC) - Intégration système profonde (Apple Neural Engine dédié)	- Exclusif iOS 18+ / iPhone 15 Pro et supérieur - Totalement fermé (code source inaccessible, pas d'API publique) - Pas d'adaptation possible par les développeurs tiers - Aucune reproductibilité pour la recherche académique

[16]Modèle

MobileLLM — Sub-milliard

Liu et al. — Meta AI — ICML 2024 · arXiv:2402.14905

125M à 1B paramètres pour appareils d'entrée de gamme. L'architecture prime sur les données à cette échelle : modèle profond et fin > large et plat. Embedding sharing + layer sharing.

Avantages	Limites
- MobileLLM 1B approche LLaMA 2 7B sur les tâches d'API mobiles ~0,9 Go RAM seulement : viable même sur 4 Go de RAM total - Consommation énergétique très faible - Idéal pour suggestion de texte et auto-complétion	- Raisonnement multi-étapes très limité (GSM8K <20 %) - Support multilingue quasi inexistant - Inutilisable pour les tâches de compréhension complexe - Performances dégradées sur les instructions longues

[17]

Architecture

PowerInfer-2 — 47B sur smartphone

Xue et al. — *MobiCom 2024* · *ACM*

Exploite la sparsité des activations (~30 % neurones actifs/token). Le reste est streamé depuis le stockage flash via *neuron clustering* et pipeline I/O-calcul. Exécute TurboSparse-Mixtral 47B sur smartphone.

Avantages	Limites
- Exécute des modèles 47B sur smartphone (22× vs llama.cpp) - Brise la barrière RAM avec le stockage flash comme extension - Ouvre la voie aux modèles frontières sans cloud - Architecture réutilisable sur tout modèle sparse	- Prototype de recherche, non disponible en production - Nécessite un stockage flash rapide (NVMe) peu présent sur mobile 11,68 tok/s rest insuffisant pour un usage conversationnel fluide - Consommation énergétique élevée (accès I/O constants)

[MediaPipe]

Framework

MediaPipe LLM Inference API

Google MediaPipe — *ai.google.dev/edge/mediapipe*

Alternative flexible à ML Kit GenAI : supporte Gemma, Phi-2, Falcon 1B via LiteRT (TFLite quantifié). Android 10+ sur tout appareil avec GPU, sans appareil certifié. Pipeline Java/Kotlin/C++.

Avantages	Limites
- Compatible avec tous les appareils Android 10+ (pas de certification requise) - Supporte Gemma 2B/7B, Phi-2, Falcon 1B - Pipeline unifié clef-en-main (tokenisation + inférence + décodage) - Licence open source Apache 2.0	- Performances inférieures à AICore (pas d'accès NPU dédié) - Format LiteRT propriétaire, incompatible GGUF directement - Moins de modèles supportés que llama.cpp - Documentation moins complète que llama.cpp ou ML Kit

[18]

Benchmark

Snapdragon 8 Gen 3 — Référence SoC

Qualcomm Technologies Inc. — *Qualcomm Developer Network*

CPU Cortex-X4 3,3GHz · GPU Adreno 750 (4,7 TFLOPS FP16) · NPU Hexagon (98 TOPS INT8) · 16 Go LPDDR5X (77 GB/s). NPU accélère le prefill 50× vs CPU sur LLaMA 2 7B. Référence 2024 pour tous les benchmarks LLM mobile.

Avantages	Limites
- NPU Hexagon : prefill 50× plus rapide que CPU - Bande passante mémoire 77 GB/s (LPDDR5X) : decode très rapide - Support des modèles jusqu'à 10B en INT4 à >20 tok/s - Référence benchmarks : Xiaomi 14 Pro, Galaxy S24	- NPU Hexagon inaccessible aux frameworks open source (llama.cpp, MLC-LLM) - Propriétaire Qualcomm : HTP uniquement via SDK Qualcomm AI Engine - Potentiel NPU largement inexploité par l'écosystème open source - Prix élevé : peu représentatif du parc mobile mondial

[19][20]

Benchmark

Études COTS & Portage Mobile

Xu et al. *arXiv:2410.03613* · *Fassold CVPR Workshop 2024*

11 appareils réels testés (Xu et al.). Snapdragon 8 Gen 3 : 30–56 tok/s prefill. MLC-LLM : +20–25 % vs llama.cpp. iOS vs Android : facteur 3× d'écart. Throttling thermique 15–25 % sur milieu de gamme.

Avantages	Limites
- Référence empirique la plus complète disponible (11 SoCs réels) - Distingue les phases compute-bound vs memory-bound (lm-Meter [21]) - Valide l'avantage MLC-LLM sur GPU Vulkan - Quantifie le throttling thermique par catégorie d'appareil	- Étude pointée dans le temps : les SoCs 2025 ne sont pas couverts - Ne teste pas l'accès NPU (non exposé par llama.cpp) - Appareils milieu de gamme (<8 Go RAM) peu représentés - Absence de mesures sur iOS à échelle comparable

[22]

Architecture

LLM as a System Service

Yin, Xu & Li — arXiv preprint 2024

Vision systémique : le LLM devient un service OS mutualisé via un démon d'arrière-plan, à l'image du GPS ou de la caméra. Partagé entre toutes les applications. Réduction de 72 % de la consommation RAM globale.

Avantages	Limites
-72 % de consommation RAM système - Élimine les chargements multiples du modèle par application - Vision partiellement réalisée par AICore (Google) et Apple Intelligence - Ouvre la voie à un standard OS universel pour l'IA embarquée	- Partiellement réalisé seulement (appareils certifiés uniquement) - Pose des questions de gouvernance (quel OS contrôle l'IA ?) - Risque de monopole plateforme (Google/Apple uniquement) - Prototype académique sans implémentation open source complète

[10]

Optimisation

Synthèse quantification mobile

Anonymous Authors — arXiv:2512.06490

Analyse comparative INT8 / INT4 / INT2 / GPTQ / GGUF pour l'exécution mobile. Conclusion : **INT4 = sweet spot** pour 3-7B sur flagship. Réduction de 68,66 % validée pour LLaMA 3.2 3B sur Android (Termux + llama.cpp).

Avantages	Limites
- INT4 validé empiriquement sur appareils réels (pas seulement en théorie) - Guide opérationnel clair pour le choix de quantification - INT2 documenté comme viable pour les tâches simples (suggestion) - Base de décision pour tout déploiement mobile open source	- Auteurs anonymes : reproductibilité non vérifiée de façon indépendante - Se limite à llama.cpp (pas de comparaison MLC-LLM) - Ne couvre pas les SoCs avec NPU (Hexagon, Apple Neural Engine) - Résultats validés uniquement sur Snapdragon (pas Dimensity ni Exynos)

[3]

Framework

ML Kit GenAI APIs

Google Developers — developers.google.com/ml-kit/genai

Intégration officielle de Gemini Nano dans Android via AICore depuis 2025. APIs Kotlin/Java pour Summarization, Proofreading, Rewriting, Image Description, prompts libres. Adaptateurs QLoRA par tâche.

Avantages	Limites
- API officielle Google la plus simple à intégrer (Kotlin/Java) - Accès au NPU via AICore : performances optimales (1,5-2× vs llama.cpp) - Modèle système partagé : zéro overhead de chargement pour l'app - Adaptateurs LoRA par tâche (résumé, relecture, réécriture)	- Pixel 9/10 et Galaxy S25/S26 <i>uniquement</i> (Tensor G4/G5 ou Snap. 8 Elite) - Quota d'inférence par application (limité en fréquence) - Exécution premier plan uniquement (pas de service background) - Aucun contrôle sur le modèle sous-jacent ni sur ses mises à jour

[Flash]

Modèle

Gemini Flash & Flash-Lite

Google DeepMind — ai.google.dev/gemini-api/docs/models

Modèles cloud légers pour architecture hybride (**pas on-device**). Gemini 2.0 Flash : inférence rapide via API avec très faible latence réseau. Flash-Lite : cas d'usage haute-fréquence où coût et vitesse priment.

Avantages	Limites
- Qualité bien supérieure aux modèles on-device (modèle >>7B) - Latence réseau très faible (infra Google optimisée) - Complément Gemini Nano dans l'architecture hybride - Multimodal natif, raisonnement multi-étapes, multilingues	- Nécessite une connexion Internet (pas de mode hors-ligne) - Données envoyées vers les serveurs Google (confidentialité partielle) - Coût par requête (facturation API) - Latence réseau incompressible (>100ms) vs local (<10ms)

[23]

Architecture

Prima.cpp — Inférence distribuée

Ye et al. — arXiv:2504.08791

Répartit les couches d'un modèle 30-70B sur un cluster d'appareils Wi-Fi domestiques via parallélisme en anneau pipelined. Décodage spéculatif intégré. 674ms/token pour 70B, 26 tok/s pour 32B.

Avantages	Limites
- Modèles frontières (70B) sans cloud depuis un cluster domestique <6 % de pression mémoire par appareil - Décodage spéculatif : 26 tok/s pour un modèle 32B - Architecture extensible (ajout/retrait d'appareils dynamique)	- Nécessite plusieurs appareils ARM64 connectés en Wi-Fi stable 674ms/token pour 70B : encore trop lent pour le chat en temps réel - Prototype de recherche, pas encore en production - Latence réseau Wi-Fi variable selon l'environnement

Tendances

Modèles multimodaux & Gemma 3

Gemma Team, Google DeepMind · 2025

Gemma 3 introduit les variantes 1B, 4B et 12B avec support multimodal (vision) natif. La convergence texte/image/audio sur appareil constitue la prochaine frontière. MobileVLM et variantes visuelles de Gemma confirment cette tendance.

Avantages

Limites

- Gemma 3 4B : vision + texte on-device dans ~3,5 Go RAM
- Ouvre les cas d'usage photo/doc/OCR directement sur mobile
- Licence permissive (usage commercial autorisé)
- Architecture distillée depuis des modèles vision-langage plus grands

- Gemma 3 12B : trop lourd pour les appareils <12 Go RAM disponibles
- Qualité vision encore inférieure aux modèles cloud (GPT-4o Vision)
- Consommation énergétique plus élevée que les modèles texte seuls
- Support vidéo embarqué encore inexistant

Tableau de synthèse : Choix d'architecture selon le cas d'usage

Critère	llama.cpp	MLC-LLM	ML Kit GenAI	MediaPipe LLM	Gemini Flash
Appareils supportés	Tout ARM64	Tout + Vulkan	Flagships certifiés	Android 10+	Tous (internet)
Performance relative	1×	1,2–1,25×	1,5–2× (NPU)	1,0–1,1×	Cloud (≫)
Confidentialité	Totale	Totale	Totale (AICore)	Totale	Partielle
Hors-ligne	Oui	Oui	Oui	Oui	Non
Format modèles	GGUF universel	TVM compilé	AICore système	LiteRT/TFLite	API REST
Open source	Oui (MIT)	Oui (Apache 2)	Non	Oui (Apache 2)	Non
Facilité intégration	Moyenne	Moyenne	Très facile	Facile	Très facile
Cas d'usage idéal	R&D, prototype	Prod. GPU Mali	Apps grand public	Apps flexibles	

Références complètes : [1] arXiv:2005.14165 [4] github.com/ggml-org/llama.cpp [5] github.com/mlc-ai/mlc-llm [6] arXiv:2403.08295
[7] arXiv:2208.07339 [8] arXiv:2210.17323
[10] arXiv:2512.06490 [11] arXiv:2305.14314 [13] arXiv:2507.13575 [14] arXiv:2404.14219 [15] arXiv:2312.11805 [16] arXiv:2402.14905
[17] MobiCom 2024 [19] arXiv:2410.03613 [23] arXiv:2504.08791